Modelling Software-based Systems

Correctness by Contruction of sequential recursive algorithms using Event-B Refinement

Master Informatique

Dominique Méry
Telecom Nancy,Université de Lorraine

10 novembre 2026

dominique.mery@loria.fr

Summary

1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

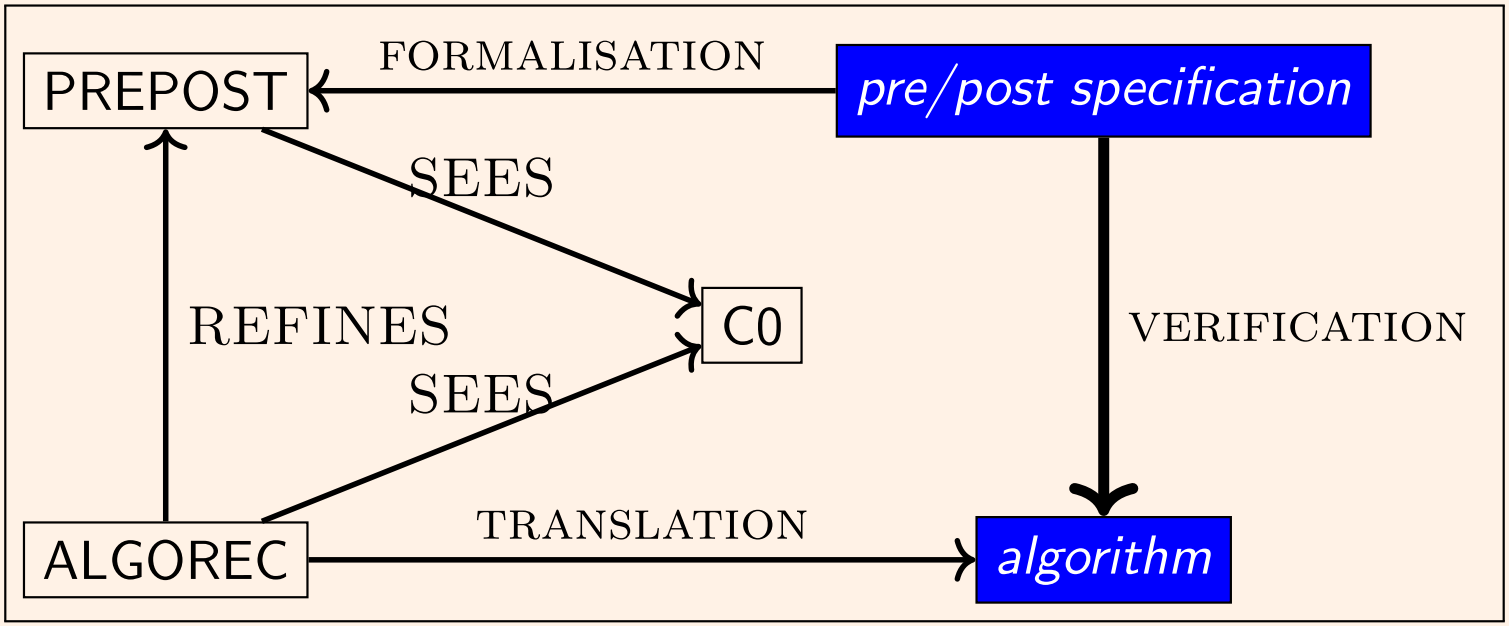
Current Section Summary

1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

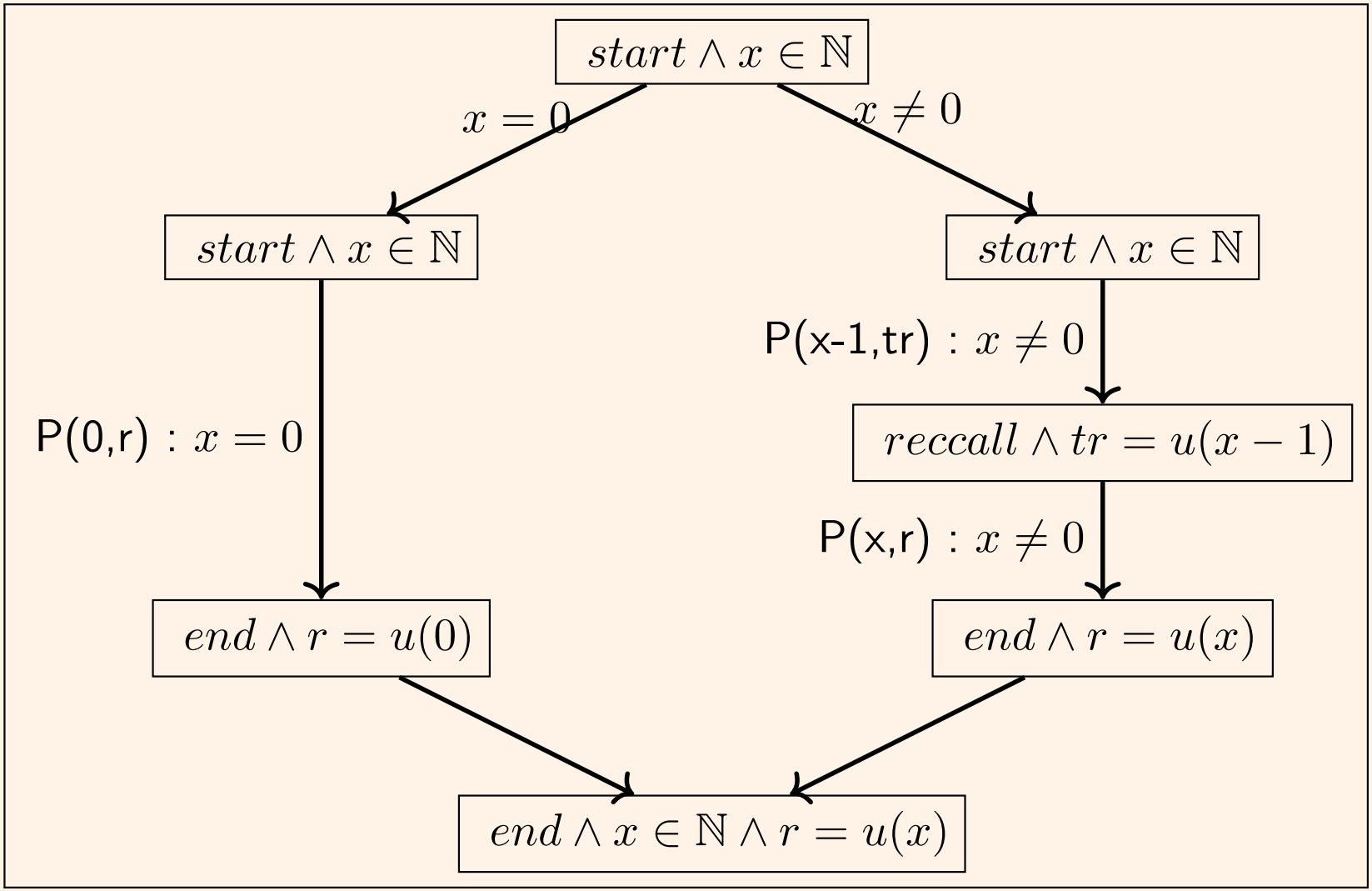
Current Section Summary

1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

The recursive pattern



The refinement diagram



Ideas for producing algorithms

- **EVENT** rec%PROC(h(x),y)%P(y) is simply simulating the recursive call of the same function.
- The invariant is defined in a simpler way by analysing the inductive structure and a control variable is introduced for structuring the inductive computation.

EVENT
e
WHERE
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x)$
THEN
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x)$
END

EVENT
rec%PROC(h(x),y)%P(y)
ANY y
WHERE
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
THEN
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
END

EVENT
call%APROC(h(x),y)%P(y)
ANY y
WHERE
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
THEN
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
END

The computation of the function x^2

VARIABLES

r, c, tr

INVARIANTS

$inv1 : r \in \mathbb{N}$

$inv2 : c = end \Rightarrow r = n * n$

$inv3 : c = callrec \Rightarrow n \neq 0$

$inv4 : c = callrec \Rightarrow tr = (n-1) * (n-1)$

$inv5 : c \in C$

$inv6 : tr \in \mathbb{N}$

$inv7 : c = end \Rightarrow r = n * n$

$inv8 : c = end \wedge n \neq 0$
$\Rightarrow tr = (n-1) * (n-1) \wedge r = tr + 2 * (n-1) + 1$

$inv9 : c = callrec \Rightarrow n * n = tr + 2 * (n-1) + 1$

The computation of the function x^2

EVENT INITIALISATION
BEGIN
$act1 : r := 0$
$act2 : c := start$
$act3 : tr :\in \mathbb{N}$
END
EVENT square0
REFINES square(n ;r)
WHEN
$grd1 : c = start$
$grd2 : n = 0$
THEN
$act1 : c := end$
$act2 : r := 0$
END

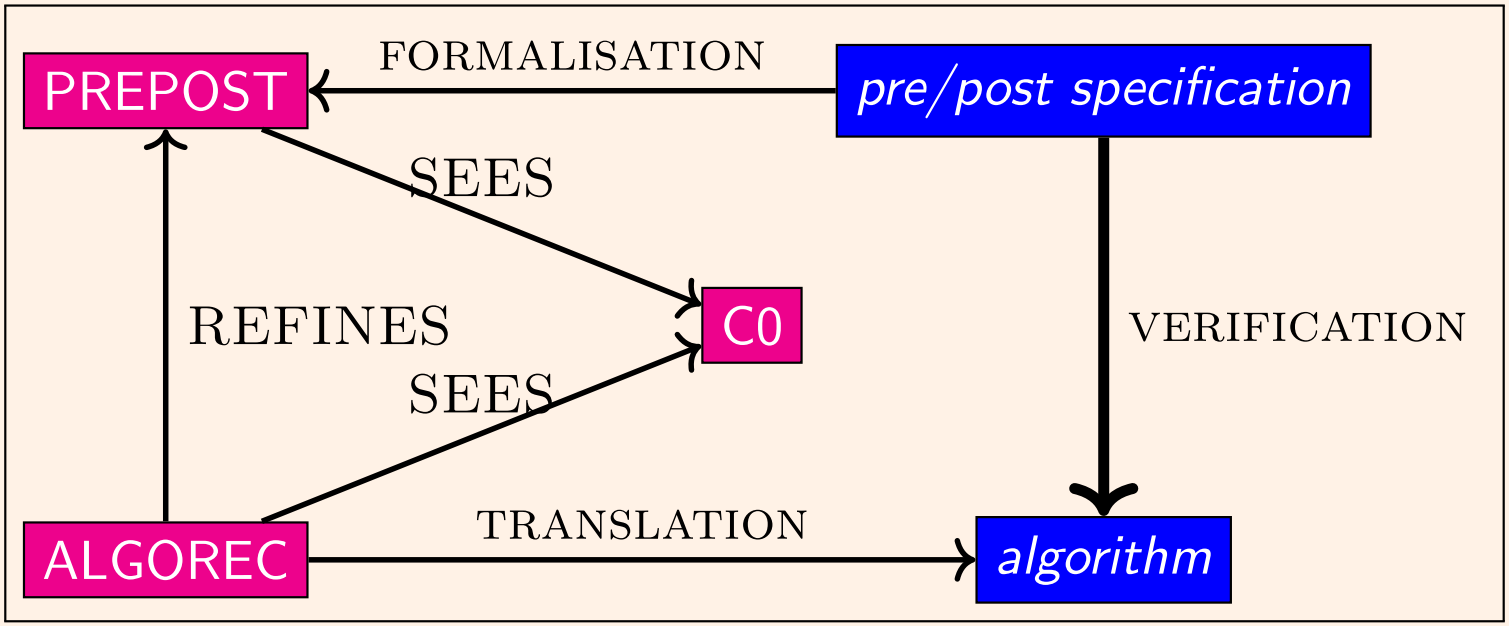
EVENT squaren
REFINES square(n ;r)
WHEN
$grd1 : c = callrec$
THEN
$act1 : r := tr + 2 * (n - 1) + 1$
$act2 : c := end$
END
EVENT rec%square(n-1 ;tr)
WHEN
$grd1 : c = start$
$grd2 : n \neq 0$
THEN
$act1 : c := callrec$
$act2 : tr := (n - 1) * (n - 1)$
END

Summary for proof obligations

Summary for proof obligations

Name	Total	Automatic	Interactive
cae-square	34	32	2
square0	3	2	1
specquare	2	2	0
square	29	28	1

The recursive pattern



Comments

- Proofs are easier and simpler
- Invariant is simple to find
- Translation is automatic
- Students are not happy with it and tools are not always set for this verification.

Current Section Summary

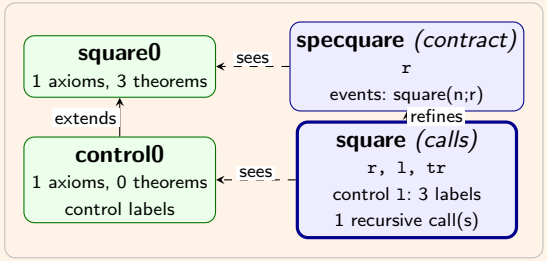
1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

Current Sub Section Summary

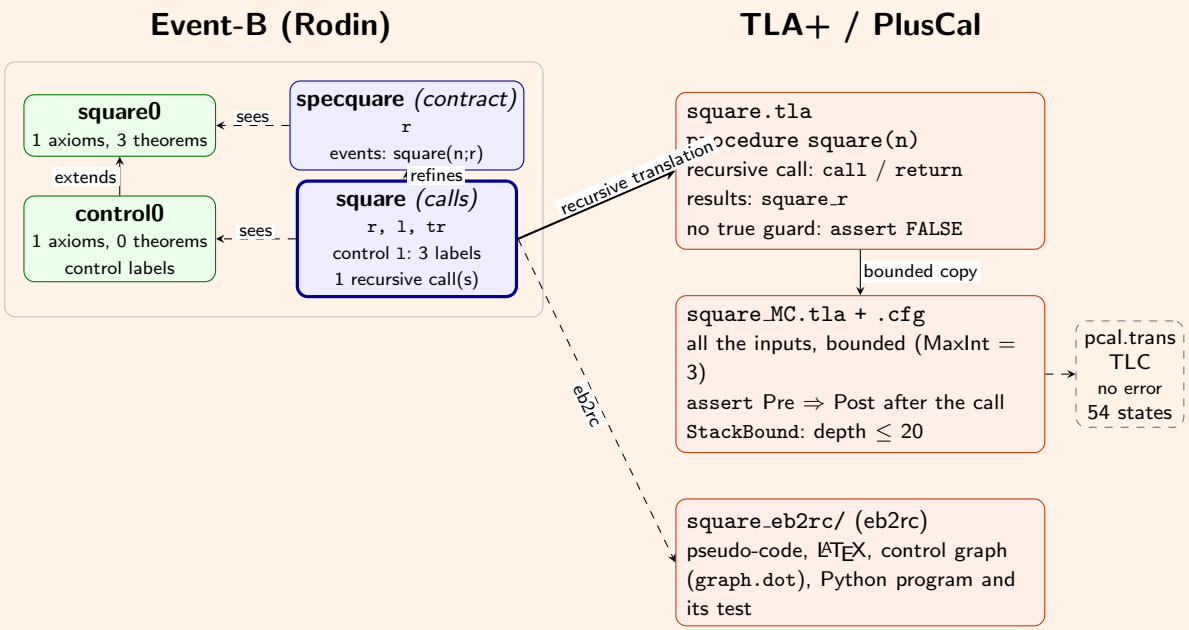
1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

Global view of the development

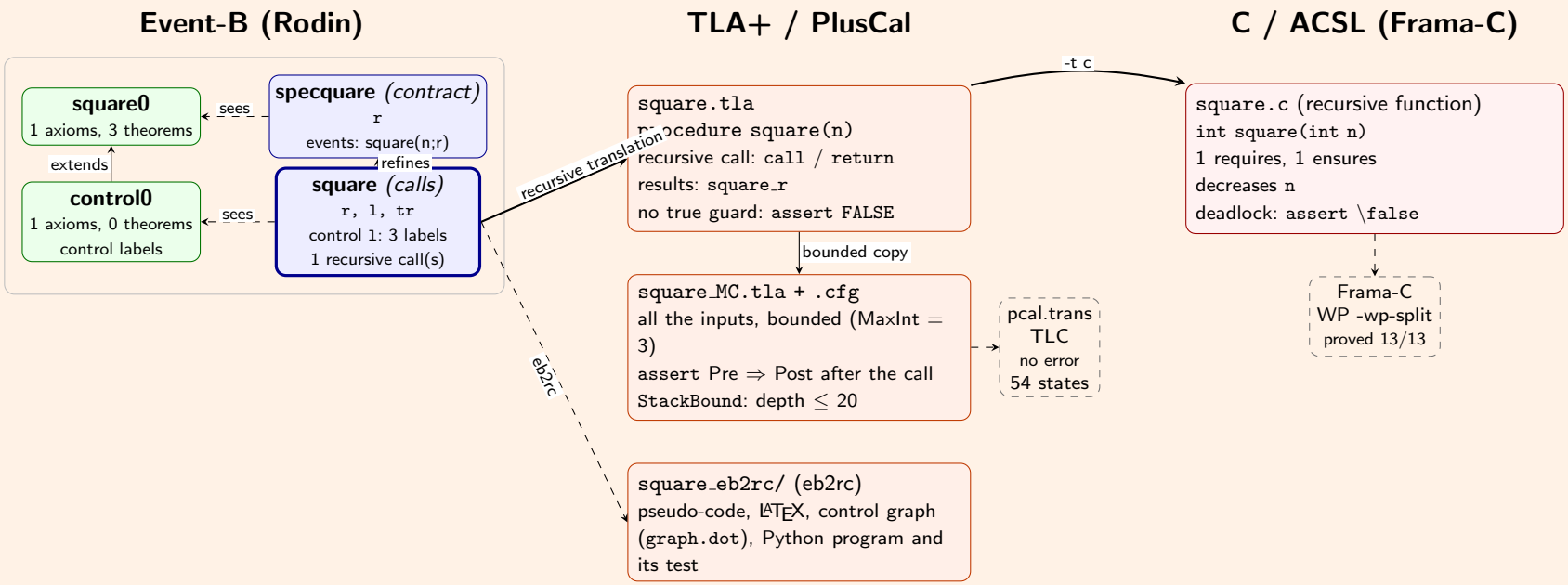
Event-B (Rodin)



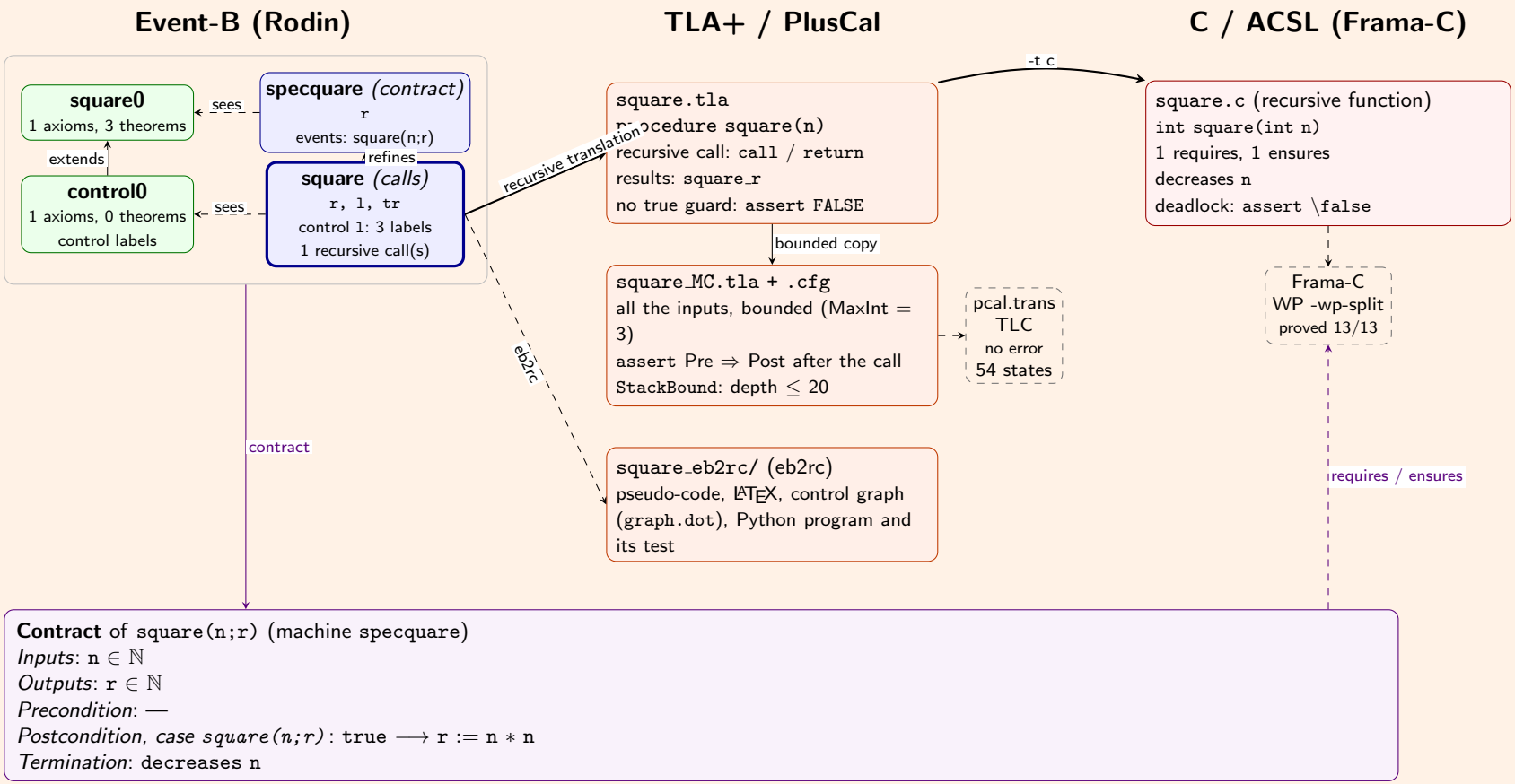
Global view of the development



Global view of the development



Global view of the development



Code for the function

```
/* Recursive function square(n;r) translated by eb2pcal (case rec) from the
   machine square of the project zb-square; contract: machine specquare.
   Check with: frama-c -wp -wp-split -wp-prover alt-ergo,z3 square.c */

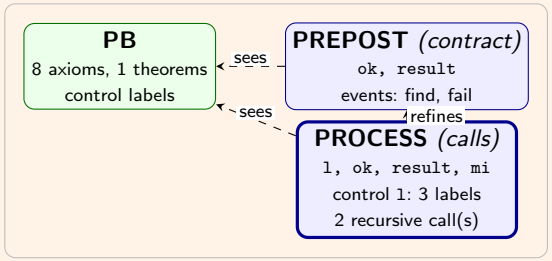
/*@
  requires (0 <= n);    // typage de n
  decreases n;
  assigns \nothing;
  ensures ((\result == (n * n)));
  // square(n;r)
*/
int square(int n)
{
    int tr = 0;
    int r = 0;
    if ((n == 0)) {
        r = 0;
    } else if ((n != 0)) {
        tr = square((n - 1));
        r = ((tr + (2 * (n - 1))) + 1);
    } else {
        /*@ assert \false; */  /* no guard true: deadlock */
    }
    return r;
}
```

Current Sub Section Summary

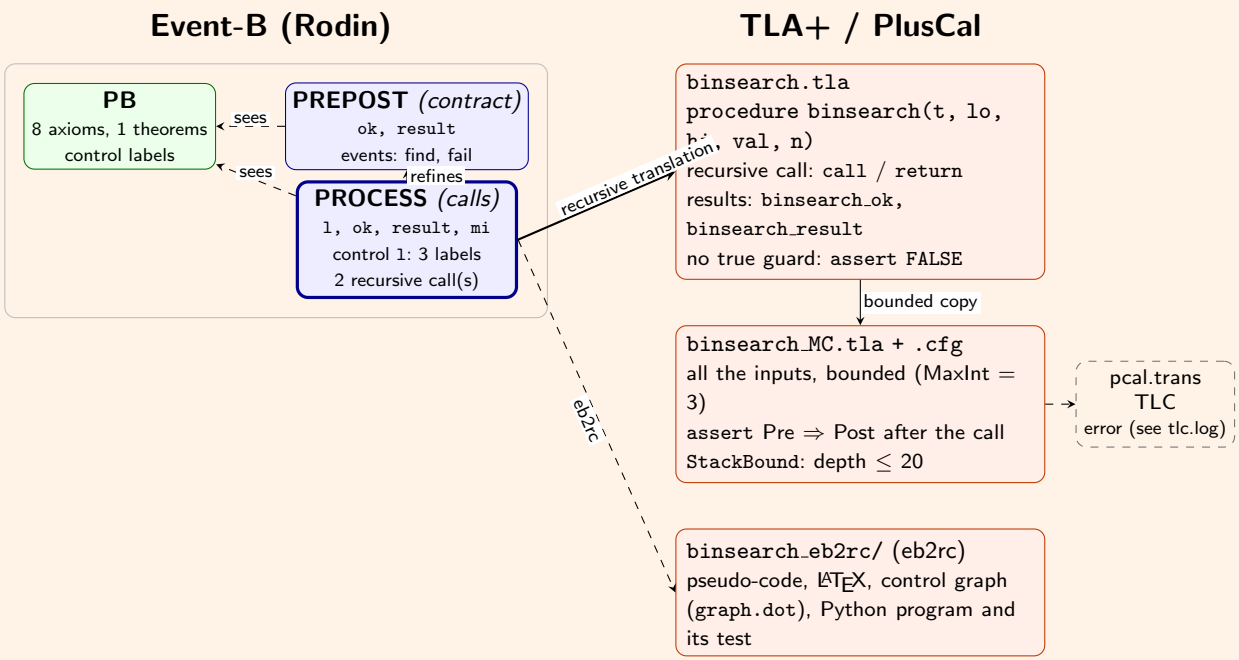
1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

Global view of the development

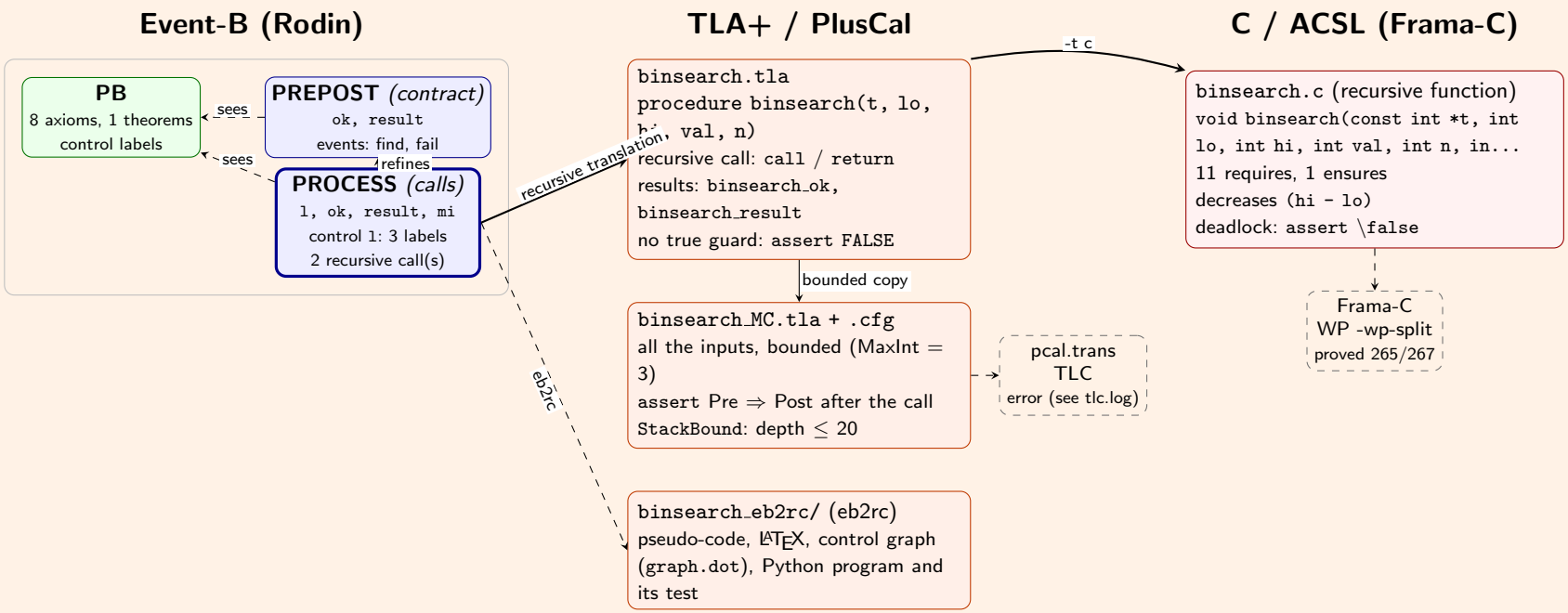
Event-B (Rodin)



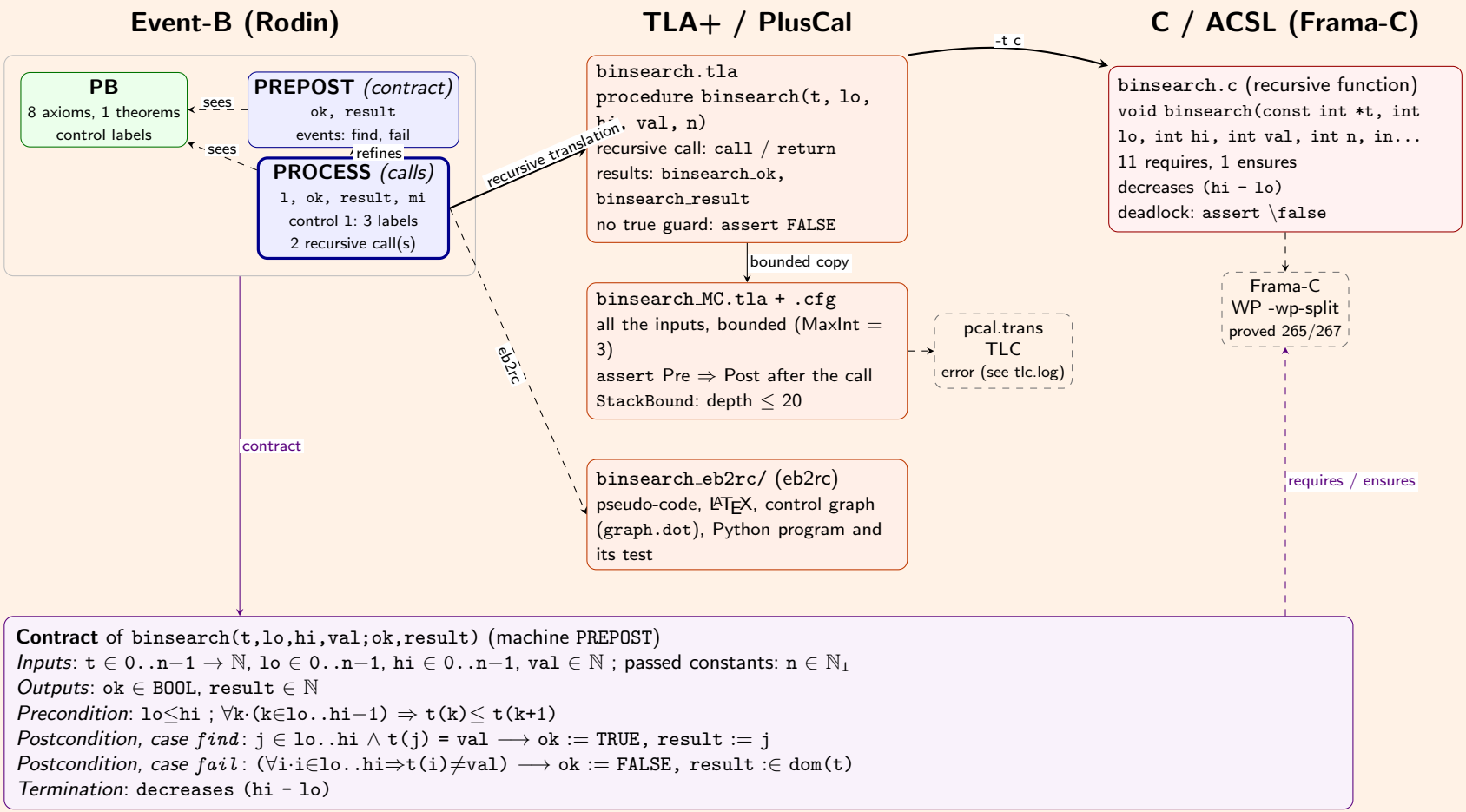
Global view of the development



Global view of the development



Global view of the development



Code for the function

```
/* Recursive function binsearch(t,lo,hi,val;ok,result) translated by eb2pca
   machine PROCESS of the project zb-binarysearch; contract: machine PREPOS
   Check with: frama-c -wp -wp-split -wp-prover alt-ergo,z3 binsearch.c */

/*@ lemma PB_axm7_global{L}:   // RR13 : Ã  prouver par rÃ©currence
      \forall int *t, integer a, b;
        (\forall integer k; a <= k <= b - 1 ==> t[k] <= t[k + 1]) ==>
        \forall integer i, j; a <= i <= j <= b ==> t[i] <= t[j];
*/

/*@
  requires \valid_read(t + (0 .. (n - 1)));   //
  requires \forall integer k_; 0 <= k_ <= (n - 1) ==> (0 <= t[k_]);
//
  requires (0 <= lo && lo <= (n - 1));   // typage de lo
  requires (0 <= hi && hi <= (n - 1));   // typage de hi
  requires (0 <= val);   // typage de val
  requires (1 <= n);   // typage de n
  requires (lo <= hi);   // PB.axm6
  requires (\forall integer k; ((lo <= k && k <= (hi - 1)) ==> (t[k] <= t[(
// PB.axm7
  requires \forall integer i_, j_; lo <= i_ <= j_ <= hi ==> t[i_] <= t[j_];
// PB.axm7 (forme globale, RR13)
  requires \valid(ok) && \valid(result);   // sorties
  requires \separated(ok, result, t + (0 .. (n - 1)));   // sorties sÃ©parÃ
  decreases (hi - lo);
  assigns *ok, *result;
  ensures (\exists integer j; (lo <= j && j <= hi) && (t[j] == val) && (*ok
       || ((\forall integer i; ((lo <= i && i <= hi) ==> (t[i] != val))) &&
// find / fail
```